

musiccloud Resolver Flow

musiccloud

Version 1.2.0+68267cd, 2026-06-06

Contents

1 Overview	2
2 Endpoints	2
3 Query Shapes	2
3.1 Streaming-Service URL	2
3.2 Free-Text Query	2
3.3 Genre-Discovery Query	2
4 Structured Search	3
4.1 Purpose	3
4.2 Syntax	3
4.3 Validation	3
4.4 Response	3
5 selectedCandidate Follow-Up	3
6 Error Cases	4
7 Practical Examples	4

1 Overview

The resolver accepts a link or search query and turns it into a unified musiccloud response. The response is either a resolved track, album, or artist, a candidate list for an ambiguous track search, a genre-discovery result, or an error with a stable error code.

This document describes the public query shapes from a user's perspective. Implementation details such as enabled plugins, admin toggles, or internal cache tables are not required to use the API.

2 Endpoints

Endpoint	Use
POST /api/v1/resolve	Full resolve endpoint for API clients with an X-API-Key or bearer token. Supports URLs, free text, genre discovery, structured search, and the selectedCandidate follow-up.
GET /api/v1/resolve	Unauthenticated scripting endpoint for one-shot track resolves. Supports URLs, free text, and structured search. Ambiguous searches return 400 because the follow-up state is not carried.

3 Query Shapes

3.1 Streaming-Service URL

Use this shape when you already have a track, album, or artist link.

```
https://open.spotify.com/track/2Wfa0iMkCvy7F5fcp2zZ8L
https://music.apple.com/de/album/example/123456789
```

The resolver detects the service, reads the source metadata, and searches for matching links on other platforms. Tracking parameters are removed before persistence and before the response is returned.

3.2 Free-Text Query

Free text is any input that is not a URL and does not start with a reserved query prefix.

```
bohemian rhapsody queen
radiohead karma police ok computer
```

This shape is useful when the user only roughly knows what they want. A high-confidence match resolves directly. If there are multiple plausible matches, POST /api/v1/resolve returns a candidate list.

3.3 Genre-Discovery Query

Genre discovery always starts with `genre:`. It is meant for browsing, not for finding one specific track.

```
genre:
genre: jazz, tracks: 20, vibe: mixed
genre: jazz|r&b, count: 5
```

`genre:?` returns a browse-grid response. `genre: <name>` returns lists for tracks, albums, and artists. Supported modifiers are `tracks`, `albums`, `artists`, `count`, and `vibe`. Use structured search instead when you are looking for a specific track.

4 Structured Search

Structured search starts with `title:`, `artist:`, or `album:`. It is meant for concrete track searches where individual fields are known.

4.1 Purpose

Structured search is more precise than free text because title, artist, and optional album can be passed to adapters separately. Services with field operators, such as Spotify, Deezer, or MusicBrainz, can search more narrowly. Other services receive the fields as combined search terms.

4.2 Syntax

`title: The Killing Moon, artist: Echo & The Bunnymen`

`artist: Radiohead`

`title: Karma Police, artist: Radiohead, album: OK Computer, count: 5`

Field	Meaning
<code>title</code>	Track title. At least one of <code>title</code> or <code>artist</code> is required.
<code>artist</code>	Artist name. At least one of <code>title</code> or <code>artist</code> is required.
<code>album</code>	Optional album title for refinement. <code>album</code> alone is rejected.
<code>count</code>	Optional integer from 1 to 10. Caps the candidate list when the search is ambiguous.

Field names are case-insensitive. Values preserve their original casing. Fields are separated by commas. A missing comma before the next known field is tolerated, for example `title: foo artist: bar`. Commas inside values are not escaped and act as field separators.

4.3 Validation

Empty values, duplicate fields, unknown fields, and `count` outside 1 to 10 return 400. Genre modifiers such as `tracks`, `albums`, `artists`, and `vibe` are intentionally rejected in structured search and belong in `genre:` queries.

4.4 Response

With high confidence, the resolver returns a track response directly. With multiple plausible matches, `POST /api/v1/resolve` returns a candidate list. `count` caps this list in addition to the server-side maximum. `GET /api/v1/resolve` cannot continue a candidate selection and returns 400 when the query is ambiguous.

5 selectedCandidate Follow-Up

When a search is ambiguous, `POST /api/v1/resolve` returns a response with `status: "disambiguation"` and candidates. Each candidate contains an opaque `id`. Send that `id` back unchanged:

```
{
  "selectedCandidate": "spotify:2Wfa0iMkCvy7F5fcp2zZ8L"
}
```

Clients should not construct the ID themselves. The format is internal and may vary by service.

6 Error Cases

Status	Typical Cause
400	Malformed query, unknown field, empty structured-search values, ambiguous GET search, or invalid <code>selectedCandidate</code> .
401	Missing or invalid credentials on protected endpoints.
404	The query is valid, but no matching track, album, or artist was found.
408	An upstream service did not answer in time.
429	Rate limit exceeded.
503	Required upstream service is temporarily unavailable.

7 Practical Examples

```
POST /api/v1/resolve
{ "query": "title: Karma Police, artist: Radiohead, count: 3" }
```

```
POST /api/v1/resolve
{ "query": "genre: jazz, tracks: 10, albums: 5" }
```

```
POST /api/v1/resolve
{ "selectedCandidate": "deezer:3135556" }
```

```
GET /api/v1/resolve
?query=title:%20Bohemian%20Rhapsody,%20artist:%20Queen
```